Phase 4: Defensive Strategies and Infrastructure Hardening

Section 4.1: Mitigating Interception Attacks with PKCE (Proof Key for Code Exchange)

The Architectural Shift from Implicit Flow to PKCE

Securing modern web and native client architectures within distributed networks requires a major architectural shift away from legacy delegation patterns.[1] The OAuth 2.0 framework originally defined the Implicit Flow to accommodate browser-based Single Page Applications (SPAs) and native mobile clients.[2] This design was a concession to early browser limitations—specifically, the absence of Cross-Origin Resource Sharing (CORS) support, which prevented client-side applications from making cross-origin HTTP POST requests to a token endpoint hosted on a different domain.[4]

To bypass this restriction, the Implicit Flow delivered Access Tokens directly in the front-channel by appending them as a URL fragment inside the HTTP redirection URI.[3] Because this flow avoids the Authorization Code exchange step, it exposes high-privilege credentials directly to the browser environment.[3] Consequently, the Implicit Flow suffers from several critical vulnerabilities:

- **Front-Channel Token Exposure:** Access tokens are exposed in the browser's address bar, rendering them vulnerable to logging by the local operating system, browser history, and proxy servers.[6]
- **HTTP Referer Header Leakage:** If the client application loads external assets or links to third-party domains, the browser may transmit the token within the Referer header.[7]
- **Cross-Site Scripting (XSS) and Script Manipulation:** Any malicious script running in the application's Document Object Model (DOM) can extract the tokens from the window location or client storage.[5]
- **Lack of Sender-Constraining:** Because the Implicit Flow lacks client authentication or a back-channel exchange, there is no standardized way to cryptographically bind the token to the legitimate client instance.[6] An intercepted token can be replayed by any attacker.[6]

To eliminate this attack surface, modern security standards, including RFC 9700 (BCP 240), deprecate the Implicit Flow in favor of the Authorization Code Flow with Proof Key for Code Exchange (PKCE, RFC 7636).[6] This hardened architecture enforces a strict separation of concerns, routing the high-privilege token delivery exclusively through a secure back-channel.[3]

Rather than delivering tokens directly via the front-channel, the Authorization Server returns a short-lived, single-use Authorization Code.[5] The client then performs an HTTP POST request to the Token Endpoint over a TLS-secured connection to exchange this code for the requested tokens.[3]

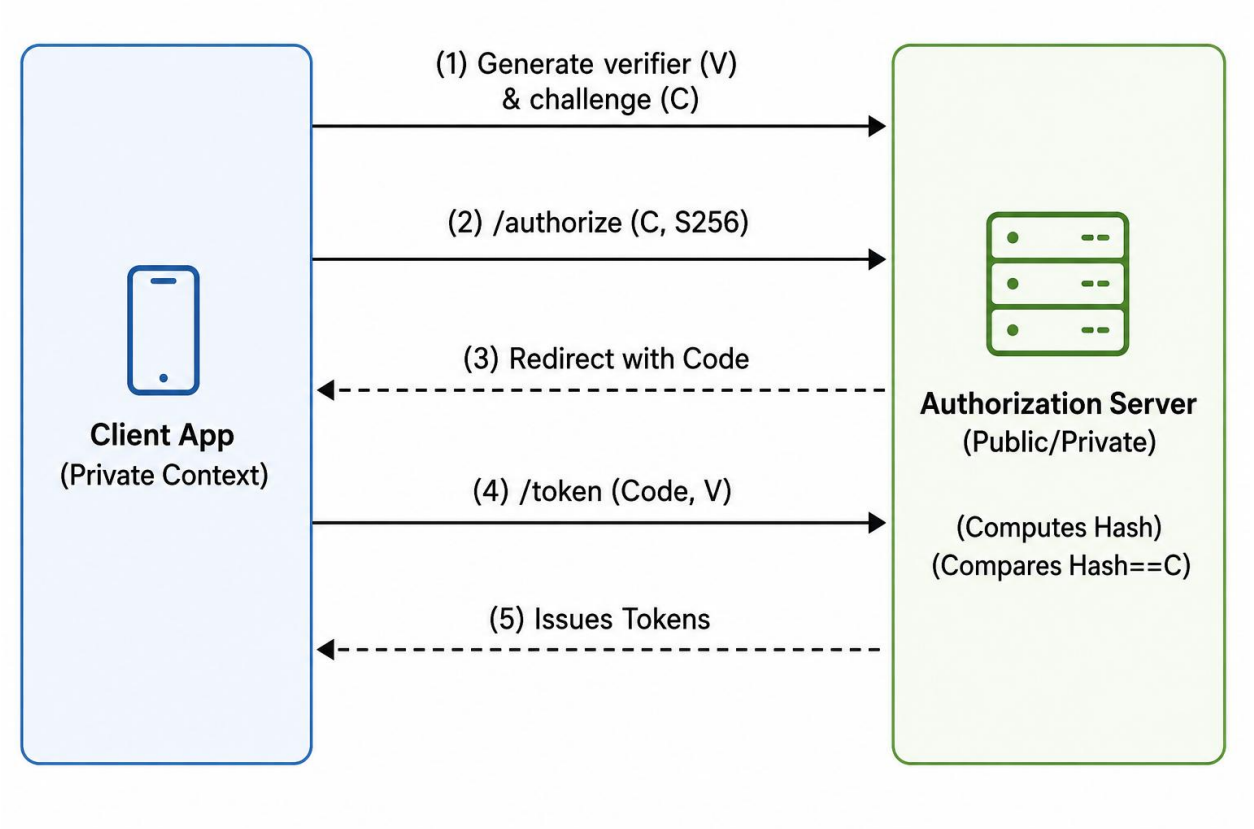
Architectural Dimension	Implicit Flow (Vulnerable	Authorization Code Flow

	State)	with PKCE (Hardened State)
Primary Token Delivery Channel	Front-channel via user agent redirection.[3]	Back-channel via direct TLS-secured POST request.[3]
Token Exposure in User Agent	High (Exposed in URL fragments and browser history).[5]	None (Tokens are returned directly in the response body of a back-channel call).[3]
Client Verification Method	Implicit trust based on pre-registered Redirect URIs.[4]	Cryptographic verification binding the authorization and token requests.[5]
Susceptibility to Interception	High (Tokens are exposed to custom URI scheme hijacking and DOM scripting).[10]	Mitigated (Intercepted codes cannot be exchanged without the client's dynamic secret).[5]
CORS Dependency	Bypassed (Relies entirely on HTTP redirects).[1]	Required (Client must perform cross-origin POST requests to the /token endpoint).[3]

This architectural change is critical for native and mobile applications.[12] Native operating systems allow multiple applications to register identical custom URI schemes (such as myapp://callback).[5] In a vulnerable state, a malicious application installed on the user's device can register the legitimate application's URI scheme, intercept the incoming redirect, extract the Authorization Code, and exchange it for an Access Token.[5] PKCE mitigates this vulnerability by introducing a dynamic, cryptographically secure verifier that links the initiation of the flow to its finalization.[5]

Cryptographic Mechanics of PKCE

The cryptographic design of PKCE ensures that only the specific client instance that initiated the authorization request can complete the token exchange.[5] This is achieved using a dynamic, high-entropy secret created on a per-request basis.[14]



Step 1: Generation of the Code Verifier

Before initiating the authorization flow, the client generates a unique, cryptographically random string known as the code_verifier (V).[3] According to RFC 7636 Section 4.1, the code_verifier must be composed of unreserved characters to prevent URL-encoding issues during transmission [15]:

$$V \in \{[A-Z],[a-z],[0-9],-,.,\_,\sim\}^{L}$$

The length L of the verifier must satisfy the constraint $43 \leq L \leq 128$, providing sufficient entropy to withstand brute-force attempts.[15] To maintain cryptographic strength, the verifier must be generated using a cryptographically secure pseudo-random number generator (CSPRNG).[15] Predictable generators, such as JavaScript's Math.random(), must not be used.[15]

Step 2: Derivation of the Code Challenge

After generating the code_verifier (V), the client derives the corresponding code_challenge (C) using a one-way cryptographic hash function.[10] RFC 7636 defines two transformation methods: plain and S256.[16]

Under the plain method, the challenge is equivalent to the verifier:

$$C_{\text{plain}} = V$$

The plain method is deprecated because it transmits the unhashed secret over the front-channel, making it vulnerable to interception by compromised TLS proxies or device logs.[6] Under the standard S256 method, the client computes the SHA-256 hash of the ASCII-encoded code_verifier and encodes the resulting binary digest using Base64URL without padding.[15] The mathematical representation of this derivation is:

$$C_{\text{S256}} = \text{Base64URL}(\text{SHA-256}(\text{ASCII}(V)))$$

This cryptographic construction leverages the pre-image resistance of SHA-256.[15] Even if an attacker intercepts the code_challenge in transit, reconstructing the original code_verifier requires a mathematically infeasible pre-image attack.[15]

Authorization Server Verification and Mitigation of Interception

The PKCE validation protocol integrates directly with the standard Authorization Code flow, verifying client authenticity at the Token Endpoint.[15]

1. Authorization Request

The client redirects the user agent to the Authorization Server's /authorize endpoint, transmitting the code_challenge (C) and the code_challenge_method (set to S256) as query parameters [15]:

```
GET /authorize?
    response_type=code
    &client_id=capstone-public-client
    &redirect_uri=myapp%3A%2F%2Fcallback
    &scope=openid%20profile%20read%3Adata
    &state=secure_csrf_token_991823
    &code_challenge=E9Melhoa2OwvFrEMTJguCHaoeK1t8URWbuGJSstw-cM
    &code_challenge_method=S256 HTTP/1.1
Host: identity.distributed-network.net
```

The Authorization Server persists the incoming code_challenge and code_challenge_method in its session store, binding them to the issued Authorization Code.[15]

2. Token Exchange and Verification

Upon receiving the Authorization Code via the redirect URI, the client issues an HTTP POST request to the /token endpoint.[15] It includes the raw code_verifier (V) to prove ownership of the transaction [14]:

```
POST /token HTTP/1.1
Host: identity.distributed-network.net
Content-Type: application/x-www-form-urlencoded

grant_type=authorization_code
&client_id=capstone-public-client
```

```
&code=SplxlOBeZQQYbYS6WxSbIA
&redirect_uri=myapp%3A%2F%2Fcallback
&code_verifier=dBjftJeZ4CVP-mB92K27uhbUJU1p1r_wW1gFWFOEjXk
```

Upon receiving this request, the Authorization Server executes the following validation steps:

1. It retrieves the persisted code_challenge and code_challenge_method associated with the presented Authorization Code.[15]
2. It verifies the presence of the code_verifier parameter in the token request.[16]
3. It computes the cryptographic hash of the incoming code_verifier using the algorithm specified by the registered code_challenge_method (SHA-256 for S256).[15]
4. It performs a constant-time string comparison between the newly computed hash and the stored code_challenge.[15]

If the values match, the server verifies that the client requesting the tokens is the same client that initiated the flow.[5] If an attacker intercepts the Authorization Code, they cannot exchange it.[5] Because the attacker does not possess the code_verifier (which remained in the legitimate client's memory), their token request will fail the server-side validation and be rejected with an invalid_grant error.[15]

3. Preventing PKCE Downgrade Attacks

In a PKCE downgrade attack, an attacker intercepts the authorization request and strips out the code_challenge and code_challenge_method parameters, attempting to force the server to fall back to a standard Authorization Code Flow.[6]

To mitigate this threat, RFC 9700 establishes the following requirements for Authorization Servers [6]:

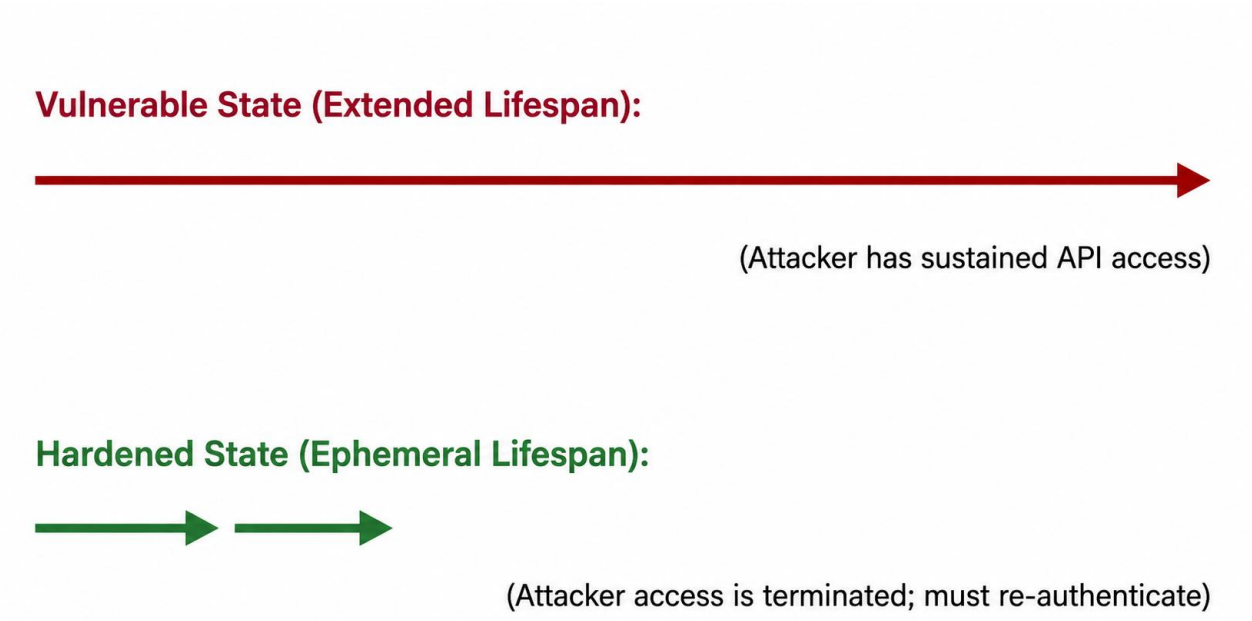
- **Mandatory Enforceability:** If a client includes a code_challenge in the authorization request, the Authorization Server MUST enforce the correct usage of the code_verifier at the token endpoint.[6]
- **Implicit Public Client Requiring:** The Authorization Server must maintain client registration metadata identifying public clients. If a public client initiates an authorization request, the server MUST reject the request if the PKCE parameters are missing.[6]
- **Verifier-Challenge Mismatch Protection:** The Authorization Server MUST reject any token request that contains a code_verifier if the corresponding authorization request did not contain a code_challenge.[6] This prevents attackers from injecting verifiers into flows that were not initialized with PKCE.[6]

Section 4.2: Attack Surface Reduction via Token Lifespan Minimization

Security Rationale Behind Short-Lived Access Tokens

Access tokens issued under the OAuth 2.0 and OpenID Connect frameworks are typically bearer tokens.[7] This means any entity that obtains the token can access the associated APIs and resource servers within the designated scopes, without providing additional credentials.[7] This model presents a risk if tokens are exfiltrated.[11] If an Access Token is assigned a long lifespan (e.g., 10 hours), a compromise of that token grants the attacker a

prolonged window of unauthorized access.[11]
To minimize this attack surface, modern security architectures reduce the Access Token lifespan to an ephemeral duration, typically between 5 and 15 minutes.[9] This mitigation serves several security objectives:



1. **Window of Opportunity Reduction:** If an Access Token is compromised, the attacker's window of opportunity is limited to the remaining lifespan of that specific token.[11]
2. **Mitigating Decentralized Revocation Latency:** In microservice-based distributed networks, Resource Servers often validate Access Tokens locally by verifying the signature of a JSON Web Token (JWT).[7] This local validation avoids the network latency of querying the central Authorization Server's /introspect endpoint for every request.[7] However, this means the Resource Server cannot detect if a token has been revoked before its expiration.[6] Assigning a short lifespan (e.g., 5 minutes) ensures that even if a token is compromised, it will naturally expire and be rejected by the microservices within minutes, providing passive revocation without complex real-time checking mechanisms.[11]
3. **Session State Synchronization:** Frequent token expiration forces the client to use a Refresh Token to obtain a new Access Token.[11] This renewal process requires the client to query the central Authorization Server, allowing the server to check the user's current session state, verify consent boundaries, and enforce policy changes in real time.[6]

Minimizing the "Window of Opportunity"

Within distributed applications, frontend-only architectures (such as browser-based SPAs) are highly vulnerable to token exfiltration.[1] Because these applications run in the user's browser, they lack a secure storage environment.[5] Storing tokens in localStorage or

sessionStorage exposes them to theft via Cross-Site Scripting (XSS) attacks.[5]
By reducing the token's lifespan to 5 minutes, the threat vector shifts from a persistent session compromise to an ephemeral breach.[17] If an attacker leverages an XSS vulnerability to exfiltrate an Access Token, they must execute their payload immediately.[11] They cannot use the token for long-term data exfiltration or state modification, as any request sent after the 5-minute threshold will be rejected by the Resource Server.[11]
This mitigation also reduces risks associated with network interception.[5] While Transport Layer Security (TLS) secures data in transit, tokens can leak at decryption points, such as TLS-terminating reverse proxies, corporate firewalls, or load balancers.[5] A stolen, short-lived token minimizes the utility of these intercepted credentials, preventing long-term replay attacks against back-end APIs.[6]

Integration with IETF Security Best Current Practices (BCP)

The IETF OAuth 2.0 Security Best Current Practice (BCP, published as RFC 9700) outlines token lifetime management requirements.[6] RFC 9700 recognizes that access token leakage is a primary threat in distributed networks, particularly when public clients are involved.[6]
To secure implementations, the BCP mandates specific token handling practices:

1. **Sender-Constraining:** The BCP recommends that Access Tokens be sender-constrained using mechanisms such as mutual TLS (mTLS, RFC 8705) or Demonstrating Proof-of-Possession (DPoP, RFC 9449).[6] These protocols require the sender to prove possession of a private key during the API call, preventing stolen tokens from being replayed by other entities.[6]
2. **Audience Restriction:** Under RFC 9700, Access Tokens must be audience-restricted.[6] This limits the token's validity to a specific Resource Server, preventing a malicious resource server from replaying a token to access other services.[6]
3. **Coexistence with Refresh Tokens:** Because short-lived tokens require frequent renewals, clients rely on Refresh Tokens to obtain new Access Tokens without manual user re-authentication.[11] To secure this model, the BCP mandates that when Refresh Tokens are issued to public clients, they must either be sender-constrained or protected by Refresh Token Rotation.[6]

Security Vector	Vulnerable State (Extended Lifespan)	Hardened State (Ephemeral Lifespan)
Typical Lifespan	8 to 12 hours [7]	5 to 15 minutes [17]
Storage Dependency	High (Often persisted to disk, local storage) [9]	Low (Stored strictly in memory or secure context) [9]
Window of Exposure	Prolonged (Sustained unauthorized API access) [11]	Restricted (Self-limiting window) [11]

Compromise Mitigation	Requires active CRL lookup or token revocation [6]	Passive expiration diminishes value of exfiltrated token [11]
Client Handling complexity	Minimal (Infrequent authorization calls) [11]	Elevated (Requires seamless refresh handling) [11]

Section 4.3: Implementing Refresh Token Rotation and Revocation

Refresh Token Rotation as a Defense-in-Depth Mechanism

While short-lived Access Tokens minimize the impact of token exfiltration, they require a mechanism to renew access without disrupting the user experience.[11] Refresh Tokens provide this capability by allowing clients to request new Access Tokens directly from the token endpoint.[7] However, Refresh Tokens are highly sensitive credentials because they typically have long lifespans (ranging from days to months).[11] If an attacker exfiltrates a Refresh Token, they can repeatedly generate new Access Tokens, gaining long-term unauthorized access.[6]

To address this threat, RFC 9700 establishes Refresh Token Rotation (RTR) as a mandatory defense-in-depth mechanism for public clients that utilize refresh tokens.[6] Refresh Token Rotation eliminates the static nature of Refresh Tokens.[6] Instead of using a single, long-lived token, the client receives a dynamic, single-use Refresh Token that is replaced during each renewal cycle.[6]

Operational Lifecycle Flow of RTR

Under Refresh Token Rotation, the lifecycle of a token exchange follows a strict state transition model:

1. **Initial Issue:** Upon successful user authentication and authorization code exchange, the Authorization Server issues an initial Access Token (AT_1) and a cryptographically random Refresh Token (RT_1).[3]
2. **Refresh Invocation:** When AT_1 approaches expiration, the client sends a token request to the /token endpoint, presenting RT_1.[6]
3. **Server Validation and Invalidation:** The Authorization Server validates the signature, database state, and client binding of RT_1.[6] If valid, the server immediately invalidates RT_1, marking its status as revoked in the database.[6]
4. **New Token Pair Generation:** The server generates a new Access Token (AT_2) and a new Refresh Token (RT_2), linking RT_2 to the same authorization grant family as RT_1.[6]

5. **Payload Response:** The server returns AT_2 and RT_2 to the client.[6] The client discards the invalidated RT_1 and stores RT_2 for the next refresh cycle.[6]

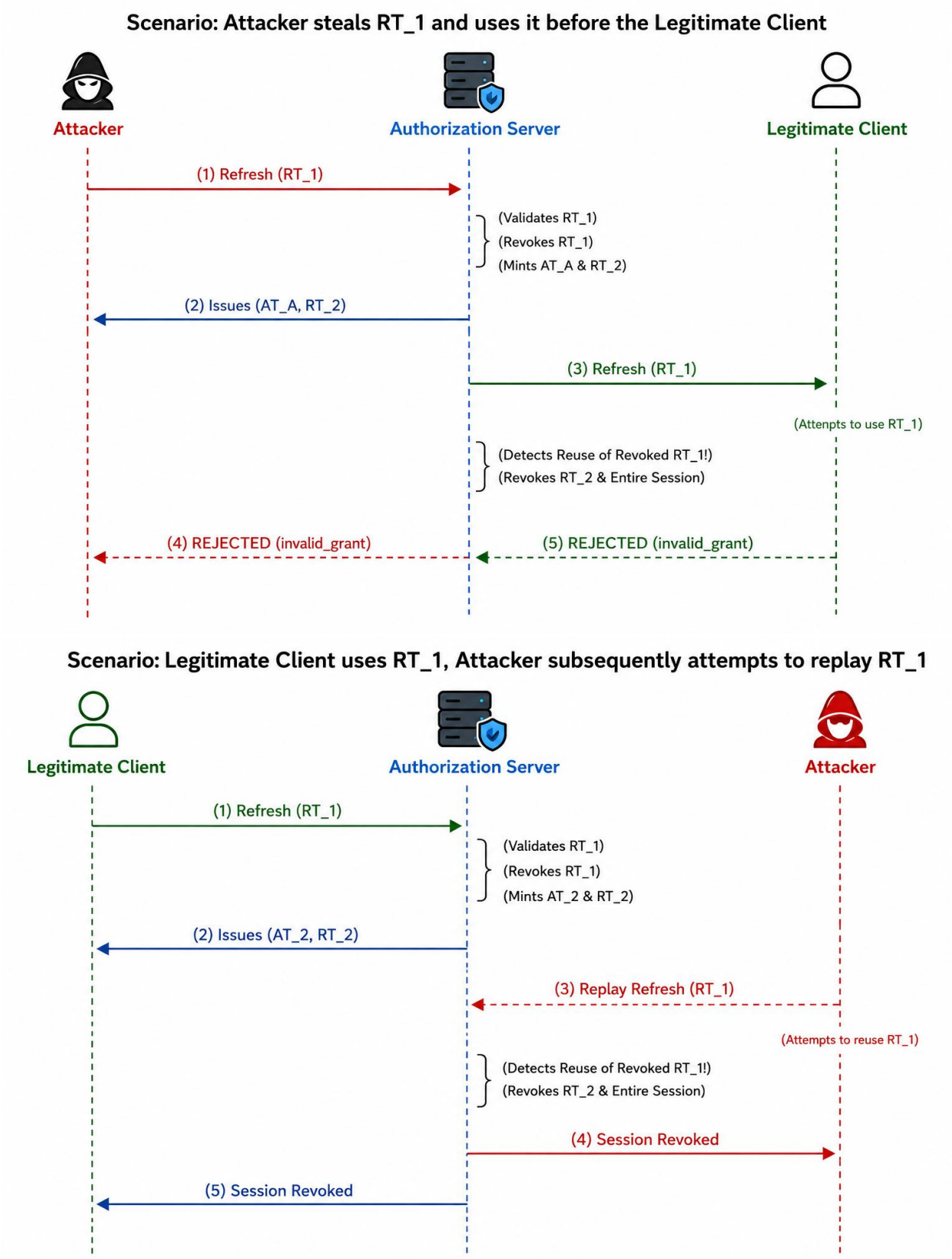
This cycle repeats for every subsequent refresh request, ensuring that any active Refresh Token is used exactly once.[6]

Threat Detection via the "Zero Reuse" Policy

The core security capability of Refresh Token Rotation is its ability to detect and mitigate token theft through a strict "Zero Reuse" policy (Refresh Token Max Reuse = 0).[6] This mechanism operates under a simple premise: in a secure state, a Refresh Token is only used once by the legitimate client.[6] If the Authorization Server receives a request containing an already invalidated Refresh Token, it indicates that a replay attempt is occurring, signaling a potential breach.[6]

Because the Authorization Server cannot determine whether the legitimate client or the attacker submitted the original or the replayed token, it must assume a compromise has occurred and execute a cascade revocation [6]:

1. **Replay Detection:** The Authorization Server receives a token request presenting RT_n, which is already marked as "revoked" or "invalidated" in its datastore.[6]
2. **Cascade Revocation:** The server identifies the authorization grant family associated with RT_n.[6] It then immediately revokes the entire authorization grant.[6] This action invalidates the current active Refresh Token (RT_{n+1}), all previously issued tokens in that chain, and any associated active Access Tokens.[6]
3. **Session Termination:** The active session is terminated, forcing both the legitimate client and the attacker to re-authenticate.[6]



This state-machine architecture ensures that if a Refresh Token is compromised, the attacker's access is quickly terminated.[6] While the attacker may obtain initial access, their persistence is broken as soon as either party attempts a renewal, triggering the cascade revocation and protecting downstream resources.[6]

Additional RFC 9700 Refresh Token Security Controls

Beyond rotation, RFC 9700 defines specific architectural guidelines for securing Refresh Tokens [6]:

- **Binding to Scope and Resource Servers:** When a Refresh Token is issued, its authorization scope MUST be constrained.[6] It must be bound exclusively to the specific scopes and resource servers consented to by the resource owner during the initial authorization request.[6] This constraint prevents privilege escalation, ensuring that new Access Tokens minted by the Refresh Token cannot exceed the originally approved access boundary.[6]
- **Confidential Client Validation:** For confidential clients (e.g., server-to-server applications), the Authorization Server MUST authenticate the client before processing a refresh request.[6] The server must verify that the client presenting the Refresh Token is the exact same authenticated entity to which the token was issued.[6]
- **Automatic Expiration and Idle Timeouts:** Refresh tokens should not have infinite lifespans.[6] The Authorization Server should enforce two types of expiration policies:
  - **Absolute Lifetime:** An absolute time limit (e.g., 30 days) after which the user must re-authenticate, regardless of activity.[7]
  - **Inactivity Timeout (Sliding Window):** A sliding expiration window that invalidates the Refresh Token if it remains unused for a specified idle period (e.g., 7 days).[6]
- **Event-Driven Revocation:** The Authorization Server should automatically revoke active Refresh Tokens in response to security events, such as a user password change, account suspension, or explicit logout.[6]

Section 4.4: Hardening Reference Implementations

Client-Side Cryptographic PKCE Generation

The following client-side implementation demonstrates how to generate a cryptographically secure code_verifier and derive its corresponding code_challenge using the Web Crypto API:

```
/**
 * Generates a cryptographically secure random string
(code_verifier).
 * Conforms to RFC 7636 unreserved character set and entropy
requirements.
 *
 * @returns {string} The cryptographically secure code_verifier.
 */
function generateCodeVerifier() {
    const array = new Uint8Array(64);
    window.crypto.getRandomValues(array);
    const chars =
'ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789-
._~';
    let verifier = '';
```

```
    for (let i = 0; i < array.length; i++) {
        verifier += chars[array[i] % chars.length];
    }
    return verifier;
}

/**
 * Derives a SHA-256 Base64URL-encoded code_challenge from a
code_verifier.
 * Uses the S256 method specified in RFC 7636.
 *
 * @param {string} verifier - The plaintext code_verifier.
 * @returns {Promise<string>} The Base64URL-encoded code_challenge.
 */
async function generateCodeChallenge(verifier) {
    const encoder = new TextEncoder();
    const data = encoder.encode(verifier);
    const hashBuffer = await window.crypto.subtle.digest('SHA-256',
data);

    // Convert ArrayBuffer to binary string
    const hashArray = new Uint8Array(hashBuffer);
    let binaryString = '';
    for (let i = 0; i < hashArray.length; i++) {
        binaryString += String.fromCharCode(hashArray[i]);
    }

    // Base64URL encode without padding
    return btoa(binaryString)
       .replace(/\+/g, '-')
       .replace(/\//g, '_')
       .replace(/=+$/, '');
}
```

Server-Side Token Validation, Rotation, and Cascade Revocation

The following server-side implementation demonstrates how to process token requests, enforce PKCE validation (including downgrade protection), and manage Refresh Token Rotation with automatic replay detection:

```
const express = require('express');
const crypto = require('crypto');
const app = express();
app.use(express.urlencoded({ extended: true }));
```

```
// Mock Datastores
const clientRegistry = {
    'capstone-public-client': {
        token_endpoint_auth_method: 'none', // Public client (no
client_secret)
        require_pkce: true,
        allowed_redirect_uris: ['myapp://callback']
    }
};

const authorizationCodes = {
    // Structure:
    // 'auth_code_123': {
    //     clientId: 'capstone-public-client',
    //     redirectUri: 'myapp://callback',
    //     codeChallenge:
'E9Melhoa2OwvFrEMTJguCHaoeK1t8URWbuGJSstw-cM',
    //     codeChallengeMethod: 'S256',
    //     userId: 'user_9921',
    //     scope: 'read:data',
    //     isUsed: false
    // }
};

const refreshTokenFamilies = {
    // Structure:
    // 'family_abc_123': {
    //     userId: 'user_9921',
    //     scope: 'read:data',
    //     isActive: true // Revoking this deactivates the entire
family
    // }
};

const refreshTokens = {
    // Structure:
    // 'hashed_token_val': {
    //     familyId: 'family_abc_123',
    //     isValid: true,
    //     expiresAt: Date.now() + 604800000 // 7 days
    // }
};
```

```
/**
 * Computes SHA-256 hash to secure tokens in database storage.
 */
function hashToken(token) {
    return crypto.createHash('sha256').update(token).digest('hex');
}

/**
 * Cryptographic helper to verify S256 PKCE challenges.
 */
function verifyPKCE(verifier, challenge) {
    const computedHash =
crypto.createHash('sha256').update(verifier, 'ascii').digest();
    const base64UrlChallenge = computedHash.toString('base64')
       .replace(/\+/g, '-')
       .replace(/\//g, '_')
       .replace(/=+$/, '');
    return crypto.timingSafeEqual(Buffer.from(base64UrlChallenge),
Buffer.from(challenge));
}

/**
 * Unified Token Endpoint handling Authorization Code Exchange and
Refresh Token Rotation.
 */
app.post('/token', async (req, res) => {
    res.setHeader('Cache-Control', 'no-store');
    res.setHeader('Pragma', 'no-cache');

    const { grant_type, client_id, code, redirect_uri,
code_verifier, refresh_token } = req.body;

    const client = clientRegistry[client_id];
    if (!client) {
        return res.status(400).json({ error: 'invalid_client' });
    }

    // --- 1. Authorization Code Exchange Flow ---
    if (grant_type === 'authorization_code') {
        const record = authorizationCodes[code];
        if (!record) {
            return res.status(400).json({ error: 'invalid_grant'
});
```

```
        }

        // Prevent code reuse
        if (record.isUsed) {
            console.warn(` Code reuse detected for client:
${client_id}.`);
            return res.status(400).json({ error: 'invalid_grant'
});
        }
        record.isUsed = true;

        // Redirect URI validation
        if (record.redirectUri!== redirect_uri) {
            return res.status(400).json({ error: 'invalid_grant'
});
        }

        // PKCE Validation & Downgrade Prevention
        if (client.require_pkce || record.codeChallenge) {
            if (!code_verifier) {
                return res.status(400).json({ error:
'invalid_grant', error_description: 'code_verifier is required' });
            }
            if (!record.codeChallenge) {
                // Reject if a verifier is provided but no
challenge was registered (Downgrade Defense)
                return res.status(400).json({ error:
'invalid_grant', error_description: 'PKCE mismatch' });
            }

            const isPkceValid = verifyPKCE(code_verifier,
record.codeChallenge);
            if (!isPkceValid) {
                return res.status(400).json({ error:
'invalid_grant', error_description: 'PKCE verification failed' });
            }
        }

        // Issue Initial Token Family and Rotation State
        const familyId =
`fam_${crypto.randomBytes(16).toString('hex')}`;
        refreshTokenFamilies[familyId] = {
            userId: record.userId,
```

```
            scope: record.scope,
            isActive: true
        };

        const rawRefreshToken =
crypto.randomBytes(48).toString('hex');
        const hashedRefresh = hashToken(rawRefreshToken);
        refreshTokens = {
            familyId: familyId,
            isValid: true,
            expiresAt: Date.now() + 604800000 // 7 days
        };

        const accessToken = crypto.randomBytes(32).toString('hex');

        return res.status(200).json({
            access_token: accessToken,
            token_type: 'Bearer',
            expires_in: 300, // Ephemeral Access Token: 5 Minutes
            refresh_token: rawRefreshToken,
            scope: record.scope
        });
    }

    // --- 2. Refresh Token Rotation Flow ---
    if (grant_type === 'refresh_token') {
        if (!refresh_token) {
            return res.status(400).json({ error: 'invalid_request'
});
        }

        const incomingHash = hashToken(refresh_token);
        const tokenRecord = refreshTokens[incomingHash];

        if (!tokenRecord) {
            return res.status(400).json({ error: 'invalid_grant'
});
        }

        const family = refreshTokenFamilies;

        // Threat Mitigation: Check if the entire family is revoked
or the token was already used
```

```
        if (!family.isActive ||!tokenRecord.isValid) {
            console.error(` Replay attack detected for family:
${tokenRecord.familyId}. Revoking all credentials.`);

            // Cascade Revocation: Deactivate the token family to
block future requests
            family.isActive = false;

            // Invalidate all tokens associated with this family
            Object.keys(refreshTokens).forEach(key => {
                if (refreshTokens[key].familyId ===
tokenRecord.familyId) {
                    refreshTokens[key].isValid = false;
                }
            });

            return res.status(400).json({
                error: 'invalid_grant',
                error_description: 'Token reuse detected.
Authorization has been revoked.'
            });
        }

        // Check if token has expired
        if (Date.now() > tokenRecord.expiresAt) {
            tokenRecord.isValid = false;
            return res.status(400).json({ error: 'invalid_grant',
error_description: 'Refresh token expired' });
        }

        // Rotate Tokens: Invalidate the current Refresh Token
        tokenRecord.isValid = false;

        // Generate a new Refresh Token and link it to the family
        const rawNewRefreshToken =
crypto.randomBytes(48).toString('hex');
        const hashedNewRefresh = hashToken(rawNewRefreshToken);
        refreshTokens = {
            familyId: tokenRecord.familyId,
            isValid: true,
            expiresAt: Date.now() + 604800000 // 7-day sliding
expiration
        };
```

```
        // Generate a new ephemeral Access Token
        const newAccessToken =
crypto.randomBytes(32).toString('hex');

        return res.status(200).json({
            access_token: newAccessToken,
            token_type: 'Bearer',
            expires_in: 300, // Ephemeral Access Token: 5 Minutes
            refresh_token: rawNewRefreshToken,
            scope: family.scope
        });
    }

    return res.status(400).json({ error: 'unsupported_grant_type'
});
});
```

References

1. J. Bradley, N. Sakimura, and J. Tsenn, "Proof Key for Code Exchange by OAuth Public Clients," Internet Engineering Task Force (IETF), RFC 7636, Sep. 2015.[12]
2. D. Hardt, Ed., "The OAuth 2.0 Authorization Framework," Internet Engineering Task Force (IETF), RFC 6749, Oct. 2012.[2]
3. T. Lodderstedt, J. Bradley, A. Labunets, and D. Fett, "Best Current Practice for OAuth 2.0 Security," Internet Engineering Task Force (IETF), RFC 9700 / BCP 240, Jan. 2025.[13]

Works cited

1. From Implicit to Authorization Code with PKCE & BFF | by Kirill Kovalev | Abblix Insights on Authentication and OpenID Connect | Medium, accessed May 28, 2026, https://medium.com/abblix-insights-on-authentication-and-openid/from-implicit-to-authorization-code-with-pkce-bff-c161b9e59b6a
2. OAuth 2.0 for Browser-Based Apps - IETF, accessed May 28, 2026, https://www.ietf.org/archive/id/draft-ietf-oauth-browser-based-apps-13.html
3. Implicit flow vs. Authorization code flow: Why implicit flow is dead? - Logto blog, accessed May 28, 2026, https://blog.logto.io/implicit-flow-is-dead
4. Is authorization code flow with public client secret equivalent to implicit flow? - Stack Overflow, accessed May 28, 2026, https://stackoverflow.com/questions/78621193/is-authorization-code-flow-with-public-client-secret-equivalent-to-implicit-flow
5. From the Implicit flow to PKCE: A look at OAuth 2.0 in SPAs - Pragmatic Web Security, accessed May 28, 2026, https://pragmaticwebsecurity.com/articles/oauthoidc/from-implicit-to-pkce.html
6. RFC 9700 - Best Current Practice for OAuth 2.0 Security - IETF Datatracker, accessed May 28, 2026, https://datatracker.ietf.org/doc/html/rfc9700

7. RFC 6749 Deep Dive: Understanding OAuth 2.0 Design Decisions from the Specification, accessed May 28, 2026, https://dev.to/kanywst/rfc-6749-deep-dive-understanding-oauth-20-design-decisions-from-the-specification-2amb
8. Authorizing Third-Party Applications Served through Messaging Platforms - PMC, accessed May 28, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC8433987/
9. OAuth 2.0 Explained: A Comprehensive Security Perspective for Modern Identity Governance - Avatier, accessed May 28, 2026, https://www.avatier.com/blog/oauth-explained-identity-governance/
10. Implicit Flow vs. Code Flow with PKCE - Christian Lüdemann, accessed May 28, 2026, https://christianlydemann.com/implicit-flow-vs-code-flow-with-pkce/
11. What Is OAuth 2.0? OAuth 2 Explained for Developers - Bruno Blog, accessed May 28, 2026, https://blog.usebruno.com/what-is-oauth-2.0-oauth-2-explained-for-developers
12. RFC 7636 - Proof Key for Code Exchange by OAuth Public Clients - IETF Datatracker, accessed May 28, 2026, https://datatracker.ietf.org/doc/html/rfc7636
13. RFC 9700 - Best Current Practice for OAuth 2.0 Security - IETF Datatracker, accessed May 28, 2026, https://datatracker.ietf.org/doc/rfc9700/
14. OAuth PKCE code_verifier reused as state parameter — verifier leaked via authorization URL · Issue #10693 · NousResearch/hermes-agent - GitHub, accessed May 28, 2026, https://github.com/NousResearch/hermes-agent/issues/10693
15. RFC 7636 Deep Dive: How PKCE Kills Authorization Code Interception Attacks, accessed May 28, 2026, https://dev.to/kanywst/rfc-7636-deep-dive-how-pkce-kills-authorization-code-interception-attacks-91i
16. Proof Key for Code Exchange (RFC 7636) - Authlete, accessed May 28, 2026, https://www.authlete.com/developers/pkce/
17. A Critical Analysis of Refresh Token Rotation in Single-page Applications | Ping Identity, accessed May 28, 2026, https://www.pingidentity.com/en/resources/blog/post/refresh-token-rotation-spa.html
18. 5 Steps to Implement OAuth 2.0 in Banking APIs - Lucid.now, accessed May 28, 2026, https://www.lucid.now/blog/implement-oauth-2-0-banking-apis-steps/
19. Updates to OAuth 2.0 Security Best Current Practice - IETF Datatracker, accessed May 28, 2026, https://datatracker.ietf.org/doc/draft-ietf-oauth-security-topics-update/